coursera | UNIVERSITY OF MICHIGAN

Applied Information Extraction in Python

Today's goal • 3/3 items

Module 1
Information Extraction

Module 2
Named Entity Recognition (NER)

Module 3
Neural Network Models

What Is Deep Learning?

- Introduction to Deep Learning
  Video • 4 min
- Understanding Deep Learning: A Shift From Rules to Representation
  Reading • 10 min
- Knowledge Check: What Is Deep Learning?
  Practice Assignment • Grade: 100%

What Are Neural Network Models?

- Neural Network Models
  Video • 7 min
- Activation Functions: How Deep Learning Models Make Decisions
  Reading • 10 min
- Knowledge Check: What Are Neural Network Models?
  Practice Assignment • Grade: 100%

Deeper Neural Networks

- Deep Neural Network Models
  Video • 6 min
- **Understanding Deep Neural Network Models: How Depth Enables Learning at Scale**
  Reading • 10 min
- Knowledge Check: Deeper Neural Networks
  Practice Assignment • 10 min

How to Build IE Models Using DNNs

- Building an Information Extraction Pipeline with BiLSTMs and CRFs
  Reading • 10 min
- Jupyter Notebook Practice on Training LSTMs
  Lab • 1h
- Demo: Configuring the Bi-Directional LSTM
  Video • 4 min

Module Wrap-Up & Assignment

- Assignment 3 Introduction: Building an Information Extraction Pipeline with BiLSTMs and CRFs
  Video • 3 min
- Build an Information Extraction Pipeline using Deep Neural Networks
  Programming Assignment • 3h
- Module 3 Assignment
  Graded Assignment • 30 min

Module 4
Transformers Transform Information Extraction

Understanding Deep Neural Network Models: How Depth Enables Learning at Scale

0:32 / 4:42 2x

Why Go Deep?

You've already seen how a **single neuron** works and how **activation functions** add non-linearity to neural networks. But today's most powerful models aren't built with just one or two neurons, they're built with **many layers of them**.

This is the essence of **deep learning**: models that learn by going deep.

In this reading, we'll explore:

- What makes a network "deep"
- Why stacking layers gives networks more power
- The challenge of **vanishing gradients**
- How deep architectures make **NLP tasks** like information extraction possible

What Is a Deep Neural Network?

A **Deep Neural Network (DNN)** is a model composed of:

- An **input layer** (where data enters)
- One or more **hidden layers** (where features are learned)
- An **output layer** (where predictions are made)

Each **hidden layer** consists of multiple **neurons**, and each neuron has:

- Inputs (from features or the previous layer)
- Weights
- An activation function

The **depth** of the network refers to the **number of hidden layers**. A typical DNN might have two or more hidden layers, while some modern architectures contain **dozens or even hundreds**.

Figure 1 shows a schematic of a Deep Neural Network (DNN) with h hidden layers**.**

Figure1. A Deep Neural Network (DNN) with h hidden layers

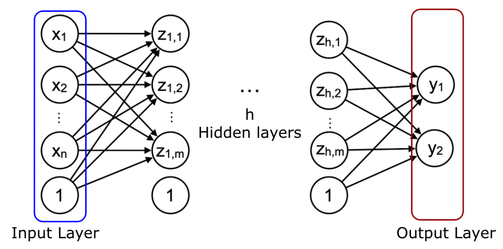


Image Description: The image shows a **diagram of a Deep Neural Network** with **three types of layers.** The layers are connected by **arrows**, indicating the **flow of information** from left to right.

Input Layer (left side, blue box): Contains neurons, shown as circles, arranged vertically. All input layer Neurons have arrows pointing to every neuron in the next layer.

Hidden Layers (middle area): The hidden layers are represented abstractly with "..." indicating h layers. The first hidden layer is shown explicitly. Similarly, the last hidden layer is shown explicitly. Each neuron in one layer is connected to every neuron in the next layer.

Output Layer (right side, red box): Contains two output neurons. Each neuron in the last hidden layer is connected to both output neurons.

Why Add More Layers?

Why not just build one large layer with lots of neurons?

Because **depth allows for hierarchical learning**.

Example from NLP: "University of Michigan Medical Center is located in Ann Arbor."

- The **first layer** might recognize individual words or tokens
- The **second layer** might detect short phrases (e.g., "University of Michigan")
- A **deeper layer** might learn that this phrase is an **organization**
- The **final layer** might assign **BIO tags** or perform **classification**

More layers = more abstraction. Each level captures patterns of increasing complexity.

What Can Go Wrong in Deeper Networks?

As powerful as deeper models are, they come with new challenges. One of the most important is the **vanishing gradient problem**.

Here's what happens:

- Activation functions like **sigmoid** flatten out for very large or small input values
- The **gradient** (which drives learning) becomes very small in early layers
- As a result:
  - Earlier layers learn very slowly or stop learning altogether
  - Most learning happens only in the final few layers
  - The network gets **"stuck"** and underperforms

This behavior is known as the **vanishing gradient** problem.

How Do We Fix It?

Researchers have developed several techniques to combat vanishing gradients:

- Use **ReLU** or **Leaky ReLU** instead of sigmoid (they don't squash gradients)
- Apply **batch normalization** to stabilize training
- Incorporate **residual connections** (e.g., in ResNet architectures) to skip layers during training

Key takeaway: Training deep networks is harder, but **worth it** when done right.

Deep Networks in Real NLP Tasks

Soon, you'll build an **information extraction pipeline** using **bi-directional LSTMs** and **CRFs**. These models are "deep" because:

- They have **multiple processing layers**
- They capture context from both **past and future** tokens
- They learn rich, internal **representations of language**

In real-world applications, deep networks power:

- Clinical concept extraction
- Automatic ICD coding
- Entity recognition from radiology or discharge summaries
- Real-time speech-to-text and transcription

Explore Further

- **CS231n – Deep Learning Architecture Overview (Stanford)**
- **Colah's Blog – Deep Learning in NLP**

Reflection Prompt

Imagine two models, one with **3 hidden layers**, the other with **20**.

What types of problems would each be best suited for? What new risks arise when your model gets very deep?

As you move into **LSTMs** and **transformers**, remember: **More layers = more expressive power**, but also more complexity to manage. Deep learning is not about stacking blindly, it's about understanding what each layer is learning.

Go to next item ✓ Completed

Like Dislike Report an issue

Go to next item →